

Spalten, Zeilen und Tabellenstrukturen bearbeiten

Section 3 · Data Transformation and Modelling · Quelle: PySpark-/Databricks-Referenzdokumentation

Neben dem Zusammenführen mehrerer Tabellen gehört das gezielte Verändern einer einzelnen Tabelle zum Kernhandwerk jeder PySpark-Transformation: Spalten hinzufügen oder entfernen, umbenennen, aufteilen, Zeilen filtern und verschachtelte Arrays auf einzelne Zeilen „auf falten“. Der Exam Guide fasst dies unter dem Ziel „Manipulate columns, rows, and table structures“ zusammen. Als Grundlage dient eine kleine, frei erfundene Bestellta be l l e:

```
bestellungen = spark.createDataFrame([
    (1, "Max Mustermann", "Kaffee,Zucker,Milch", 24.90, "2024-05-01"),
    (2, "Erika Beispiel", "Tee,Kekse", 11.50, "2024-05-03"),
    (3, "Jan Schmidt", "Kaffee", 8.90, "2024-05-04"),
], ["order_id", "customer_name", "items_raw", "amount", "order_date"])

bestellungen.createOrReplaceTempView("bestellungen")
```

1. Spalten hinzufügen: withColumn()

`withColumn(name, expr)` fügt einer bestehenden Tabelle eine neue Spalte hinzu oder überschreibt eine vorhandene Spalte gleichen Namens. Das zugrunde liegende DataFrame bleibt dabei unverändert (Spark-DataFrames sind unveränderlich, *immutable*) – `withColumn` gibt stets ein *neues* DataFrame zurück.

```
from pyspark.sql.functions import col, round as spark_round

bestellungen_erw = bestellungen.withColumn(
    "amount_mit_steuer", spark_round(col("amount") * 1.19, 2)
)
```

```
SELECT *, ROUND(amount * 1.19, 2) AS amount_mit_steuer
FROM bestellungen;
```

Sollen mehrere Spalten gleichzeitig ergänzt werden, existiert seit Spark 3.3 zusätzlich `withColumns()`, das ein Dictionary aus Spaltenname und Ausdruck entgegennimmt, statt `withColumn` mehrfach hintereinander aufzurufen.

2. Spalten entfernen: `drop()`

`drop()` entfernt eine oder mehrere Spalten aus dem DataFrame. Da hier keine Bedingung geprüft wird, ist die Methode deutlich unkomplizierter als etwa ein Filter.

```
bestellungen_ohne_rohspalte = bestellungen.drop("items_raw")
# mehrere Spalten gleichzeitig entfernen:
bestellungen.drop("items_raw", "order_date")
```

```
SELECT order_id, customer_name, amount, order_date
FROM bestellungen;
-- Alternativ dauerhaft am Tabellenschema:
ALTER TABLE bestellungen DROP COLUMN items_raw;
```

3. Spalten umbenennen: `withColumnRenamed()`

```
bestellungen_umbenannt = bestellungen.withColumnRenamed("customer_name", "kunde")
# mehrere Umbenennungen verketteten:
bestellungen.withColumnRenamed("customer_name", "kunde").withColumnRenamed("amount",
"betrag")
```

```
SELECT customer_name AS kunde, amount AS betrag
FROM bestellungen;
-- Alternativ dauerhaft am Tabellenschema:
ALTER TABLE bestellungen RENAME COLUMN customer_name TO kunde;
```

4. Spalten aufteilen: `split()`

Die Funktion `split(spalte, muster)` zerlegt eine String-Spalte anhand eines regulären Ausdrucks in ein **Array** von Teilstrings. Im Beispiel wird zum einen die kommaseparierte Artikel-Spalte `items_raw` in ein Array überführt, zum anderen der volle Kundename anhand des Leerzeichens in Vor- und Nachname aufgeteilt.

```

from pyspark.sql.functions import split

bestellungen_split = (
    bestellungen
    .withColumn("items", split(col("items_raw"), ","))
    .withColumn("vorname", split(col("customer_name"), " ").getItem(0))
    .withColumn("nachname", split(col("customer_name"), " ").getItem(1))
)

```

```

SELECT
    order_id,
    split(items_raw, ',')           AS items,
    split(customer_name, ' ')[0]   AS vorname,
    split(customer_name, ' ')[1]   AS nachname
FROM bestellungen;

```

5. Zeilen filtern: filter() / where()

`filter()` und `where()` sind in PySpark vollständig gleichwertige Aliase – welche Variante gewählt wird, ist reine Geschmackssache bzw. Lesbarkeitsfrage (viele SQL-Entwickler bevorzugen `where()`).

```

bestellungen.filter(col("amount") > 10)
bestellungen.where(col("amount") > 10)           # identisch zu filter()
bestellungen.filter("amount > 10 AND order_date >= '2024-05-03'") # als SQL-String-
Ausdruck

```

```

SELECT * FROM bestellungen
WHERE amount > 10 AND order_date >= '2024-05-03';

```

6. Arrays auffalten: explode()

`explode()` wandelt eine Array-Spalte in mehrere Zeilen um: Für jedes Element des Arrays entsteht eine eigene Ausgabezeile, während alle übrigen Spalten unverändert dupliziert werden. Das ist besonders nützlich, um aus der zuvor per `split()` erzeugten `items`-Spalte eine Zeile *pro Artikel* zu machen.

```
from pyspark.sql.functions import explode

artikel_zeilen = bestellungen_split.withColumn("item", explode(col("items")))
# Ergebnis: aus 3 Bestellungen mit insgesamt 4 Artikeln (Bestellung 1 hat 3 Artikel,
# Bestellung 2 hat 2, Bestellung 3 hat 1) werden 6 Zeilen, jeweils ein Artikel pro Zeile.
```

```
SELECT order_id, customer_name, item
FROM bestellungen_split
LATERAL VIEW explode(items) AS item;

-- Alternative moderne Schreibweise ohne LATERAL VIEW:
SELECT order_id, customer_name, explode(items) AS item
FROM bestellungen_split;
```

Enthält die Array-Spalte in manchen Zeilen `NULL` oder ein leeres Array, gehen diese Zeilen bei `explode()` vollständig verloren, da es kein Element zum Auffalten gibt. Soll die Ursprungszeile in diesem Fall dennoch (mit `NULL` in der neuen Spalte) erhalten bleiben, hilft die Variante `explode_outer()` mit identischer Syntax.

Ergänzt aus der offiziellen PySpark-Dokumentation: `pyspark.sql.functions.explode()` gibt laut Referenz „a new row for each element in the given array or map“ zurück; für leere oder `NULL`-wertige Arrays/Maps wird standardmäßig keine Zeile erzeugt – hierfür ist explizit `explode_outer()` vorgesehen. Seit Spark 3.3 ergänzt `DataFrame.withColumns()` die Einzelspalten-Methode `withColumn()` um die Möglichkeit, mehrere neue oder ersetzte Spalten in einem Aufruf über ein Dictionary zu definieren.

Quelle: [pyspark.sql.functions.explode – Apache Spark Dokumentation](#)